

MUSIC MANAGEMENT SYSTEM

ON LINKLIST

A Project Report on Implementation of
SINGLY & DOUBLY LINKED LIST in C++



DOUBLY LINKED LIST REPRESENTATION



SUBMITTED BY
MEENAKSHI MALI



DEPARTMENT
BCA



SEMESTER
2ND



SESSION
2025 – 2026

C++ NODE DEFINITION (DOUBLY LINKED LIST)

```
class Node {  
public:  
    int id;  
    string songName;  
    string artist;  
    Node* prev; // pointer to previous node  
    Node* next; // pointer to next node  
  
    // Constructor  
    Node(int _id, string _song, string _artist) {  
        id = _id;  
        songName = _song;  
        artist = _artist;  
        prev = nullptr;  
        next = nullptr;  
    }  
};
```



BUILT FOR A BETTER MUSIC EXPERIENCE



Organized Playlist
Manage songs
efficiently



Dynamic Navigation
Move forward & backward
seamlessly



Efficient Structure
Double links for better
performance



Smooth Playback Flow
Just like a real
music system



Project Report On
Music Management System
Using Link List

Submitted to

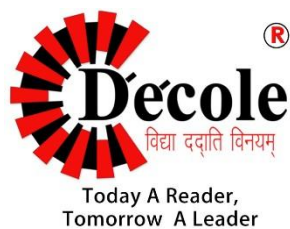
Dezyne École College

Towards the Partial Fulfilment of

BCA Semester 2nd

By

Meenakshi Mali



www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With Sincere Appreciation,

Meenakshi Mali

Department Of BCA

Dezyne École College, Ajmer

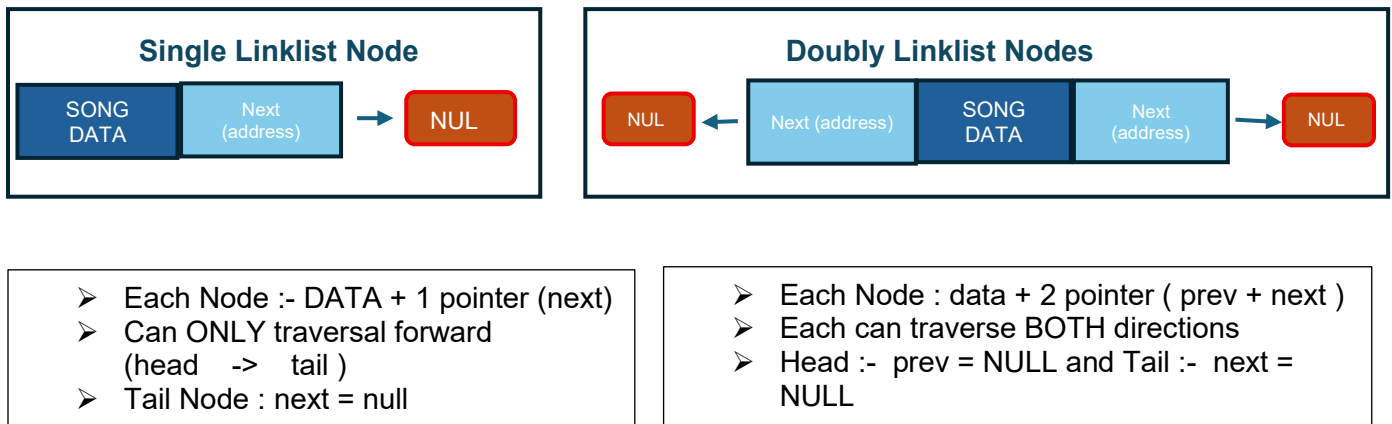
Contents

Part 1: Introduction — What is a Linked List?	1
Singly vs Doubly — At a Glance	2
Why Not an Array?.....	2
Singly Linked List — All Operations	3
Node Structure & Memory Representation	3
Insert at Head — $O(1)$	4
Insert at Tail — $O(1)$ with tail pointer	6
Insert at Position k — $O(k)$	7
Delete Operations — 3 Cases	9
Traversal — $O(n)$	13
Search by Keyword — $O(n)$	14
Count Nodes — $O(1)$ Best Practice	15
Part 2: Doubly Linked List — Insertion, Deletion & Traversal.....	16
Insert at Tail — $O(1)$	20
Insert at Position k — $O(k)$	21
Delete Head — $O(1)$	22
Delete Tail — $O(1)$ Key Advantage Over Singly.....	24
Delete given position Node — $O(1)$ with pointer / $O(n)$ if search required	25
Forward & Backward Traversal — $O(n)$ each	27

Part 1: Introduction — What is a Linked List?

A Linked List is a linear data structure where elements (called nodes) are NOT stored in contiguous memory locations. Each node holds its data and one or two pointers — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.

Real world context of – **MUSIC MANAGEMENT SYSTEM**



music management system Songs in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new song always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any song immediately — $O(1)$.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs define the **Linked List** — a data structure that does all three efficiently.

Real-World Context — MUSIC MANAGEMENT SYTEM

Twitter's home timeline shows Songs in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new Song always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any Song immediately — $O(1)$.

- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs define the **Linked List** — a data structure that does all three efficiently.

Singly vs Doubly — At a Glance

Property	Singly Linked List	Doubly Linked List
Pointers per node	1 (NEXT only)	2 (PREV + NEXT)
Traversal direction	Forward only (head → tail)	Both directions
Delete tail	$O(n)$ — must scan to 2nd-last	$O(1)$ — use tail.prev
Delete any node (with ptr)	$O(n)$ — scan for predecessor	$O(1)$ — both neighbours known
Extra memory	Less (1 ptr × n)	More (2 ptr × n = +8 bytes/node)
Typical use	Stacks, hash chaining, simple queues	Timelines, browsers, LRU cache, dequeues

Why Not an Array?

Operation	Array	Linked List	Twitter's Need
Insert at front	$O(n)$ — shift all elements right	$O(1)$ — rewire 2 pointers	New Song on top — must be instant
Delete any element	$O(n)$ — shift remaining elements	$O(1)$ — rewire neighbours	Remove any Song instantly
Access by index	$O(1)$ — direct formula	$O(n)$ — traverse from head	Not needed for sequential display
Dynamic size (grow)	Needs reallocation	Trivial — just add a node	Song count is unpredictable
Search	$O(n)$ linear, $O(\log n)$ binary	$O(n)$ linear only	Both require traversal for unsorted data

Singly Linked List — All Operations

Node Structure & Memory Representation

In a singly linked list each node stores data and ONE pointer — next — which holds the memory address of the following node. The last node's next = NULL. We maintain a head pointer to always find the start, and optionally a tail pointer for efficient end-access.

C++ struct:

Memory layout example:

```
public:
    string SID;
    string ANAME;
    string SD;
    NODE *NEXT;
    NODE(string x, string y, string z)
    {
        SID = x;
        ANAME = y;
        SD = z;
        NEXT = nullptr;
    }
};

class LinkList
{
public:
    string SID;
    int SCNT;
    string AName, SCAP;
    NODE *START,*END;
    int countNode;
    LinkList(){
        SCNT = 1001;
        countNode = 0;
        START = nullptr;
        END = nullptr;
    }
```

Insert at Head — $O(1)$

Singly linked list – Insert at Head & Insert at Tail

INSERT AT HEAD

BEFORE :-

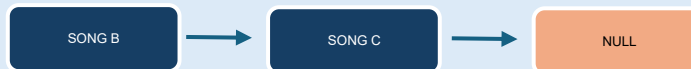


1. New node = new node (song A)



INSERT AT TAIL – $O(1)$ WITH TAIL POINTER

BEFORE :-



1. New node = new node (song C)
2. tail.next = head (= song B)
3. tail = newNode

AFTER :-



INSERT AT GIVEN POSITION

BEFORE :-



1. Curr = head :- travers 1 stepp to reach pos 1 (SONG B)
2. NewNode.next = cur.next (= SONG C at pos 2)
3. Curr.next = newNode

Concept

The newest Song always goes at the top of the timeline. Head insertion achieves this in exactly 2 pointer operations — always $O(1)$ regardless of how many Songs already exist

Step-by-Step Algorithm

1. **Create new node:** `SongNode* newNode = new SongNode(text, user);` — allocates heap memory, sets `next = NULL`.
2. **Connect forward:** `newNode->next = head;` — new node now points to the current head (will become 2nd node).
3. **Move head pointer:** `head = newNode;` — head now refers to the new node.
4. **Handle empty list:** If the list was empty, also set `tail = newNode;`
5. **Increment counter:** `size++;`

code

```
int insertNode(int opr)
{
    system("cls");
    if (opr == 1){
        cout<<"\n\nInsert SONG At Beginning . \n\n";
        getData();
        NODE *TEMP = new NODE(SID, AName, SCAP);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if (START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START = TEMP;
        }
        countNode++;
        SCNT++;
    }
}
```

Insert at Tail — $O(1)$ with tail pointer

Without tail pointer: Must traverse all n nodes to reach the last one, then append. Cost = $O(n)$.

With tail pointer (always recommended): Directly access tail, attach new node, advance tail. Cost = $O(1)$.

Step-by-Step Algorithm (with tail pointer)

1. **Create new node** `SongNode* newNode = new SongNode(text, user);` — `newNode->next = NULL`.
2. **Handle empty list:** If `head == NULL`: `head = tail = newNode;` return.
3. **Connect:** `tail->next = newNode;` — current tail now points to the new node.
4. **Advance tail:** `tail = newNode;` — tail pointer moves to the new last node.
5. **Increment counter:** `size++;`

Code

```
else if ( opr == 2){
    cout<<"\n\nInsert SONG At End . \n\n";
    getData();
    NODE *TEMP = new NODE(SID,AName, SCAP);

    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        END = TEMP;
    }
    countNode++;
    SCNT++;
}
```

Insert at Position k — $O(k)$

Three sub-cases

- **k = 0:** Same as insertAtHead() — $O(1)$.
- **k = size (last position + 1):** Same as insertAtTail() — $O(1)$.
- **0 < k < size (middle):** Traverse k-1 steps to find the predecessor node, then rewire 2 pointers.

Step-by-Step Algorithm for Middle Insert

1. Create newNode.
2. Start `curr = head`; traverse k-1 steps: `for(int i=0; i<k-1; i++) curr = curr->next;`
3. `newNode->next = curr->next;` — new node points to what curr was pointing to.
4. `curr->next = newNode;` — curr now points to new node.
5. `size++;`

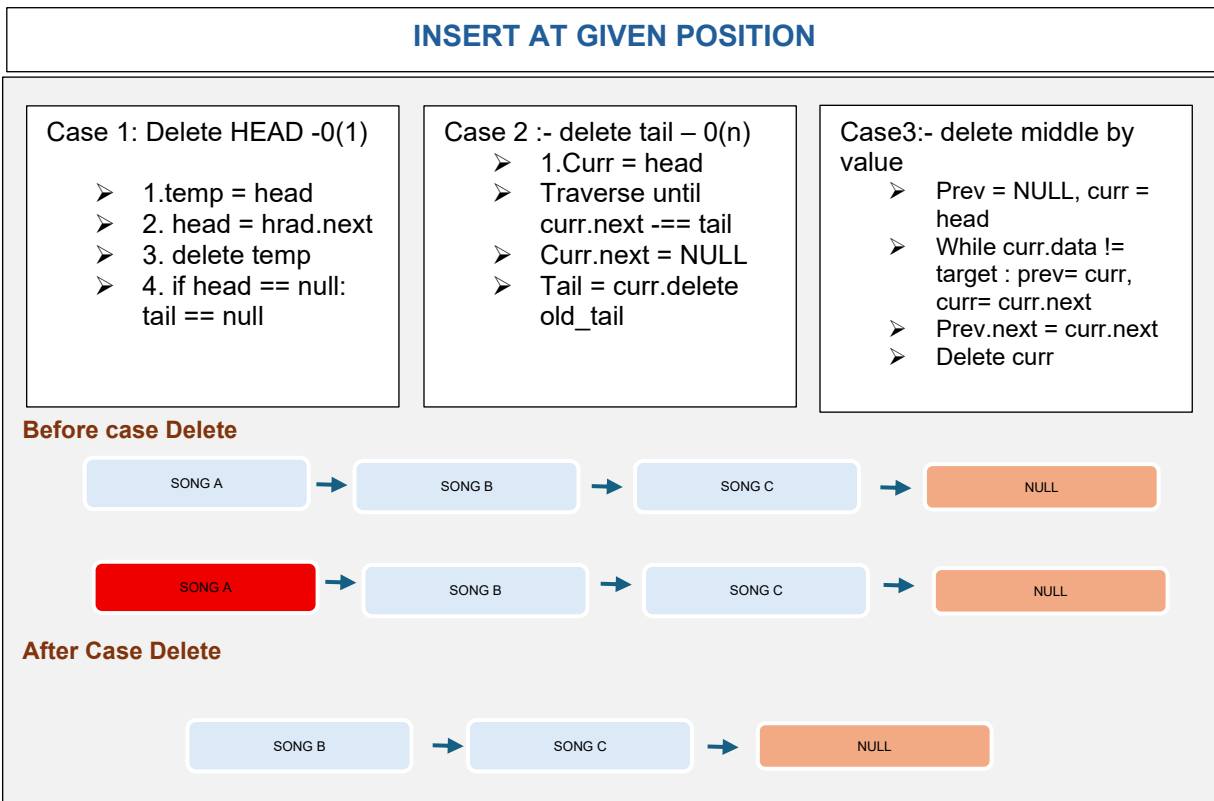
code

```
else if ( opr == 3){
    cout<<"\n\nInsert SONG At Given Position . \n\n";
    cout<<"Total Songs -- "<<countNode<<"\n\n";
    int pos;
    getData();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODE *TEMP = new NODE(SID,AName, SCAP);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
```

```
    TEMP->NEXT = TEMP1->NEXT;
    TEMP1->NEXT = TEMP;
    if ( TEMP1 == END){
        END = TEMP1->NEXT;
    }
}
countNode++;
SCNT++;
}
```

Delete Operations — 3 Cases



Case 1 — Delete Head — O(1)

1. Moving the head pointer one step forward effectively removes the first node. No traversal needed.
2. **temp = head;** — save the node to free later.
3. **head = head->next;** — head now points to the 2nd node (new head).
4. If head != NULL: it was a single-node list: set **tail = NULL;**
5. **delete temp;** — free heap memory.
6. **size--;**

```
int deleteNode(int opr)
{
    NODE *TEMP;
    if (opr == 1){
        cout<<"\n\nDelete SONG From Beginning . \n\n";
        if (START == nullptr)
        {
            cout<<"PlayList is Empty.. \n\n";
```

```

        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
    }
    countNode--;
    delete(TEMP);
    cout<<"\nSONG Deleted Successfully\n\n";
}

```

Case 2 — Delete Tail — $O(n)$ ← Singly's Weakness

Problem: To delete the tail we must update the second-to-last node's next pointer to NULL. In a singly linked list the only way to find that node is to traverse from head — $O(n)$. This is why doubly linked lists are preferred when tail deletion is frequent.

6. Start **curr = head**; traverse until **curr->next == tail**;
7. **curr->next = NULL**; — second-to-last now terminates the list.
8. **delete tail**; then **tail = curr**;
9. **size--**;

```

else if ( opr == 2){
    cout<<"\n\nDelete SONG From End . \n\n";
    if ( START == nullptr)
    {
        cout<<"PlayList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
}

```

```

else{
    TEMP = START;
    int c = 1;
    while ( TEMP->NEXT != END)
    {
        cout<<c++<<endl;
        TEMP = TEMP->NEXT;
    }
    NODE *T1 = END;
    TEMP->NEXT = nullptr;
    END = TEMP;
    cout<<END->ANAME<<endl;
    delete(T1);
}
countNode--;
cout<<"\nSONG Deleted Successfully\n\n";
}

```

Case 3 — Delete Middle Node by Given Position — $O(n)$

1. Find the target node and its predecessor simultaneously using two pointers, then bypass the target.
2. **prev = NULL; curr = head;**
3. Traverse: while curr != NULL and curr->data != target: **prev = curr; curr = curr->next;**
4. If curr == NULL: value not found — return.
5. **prev->next = curr->next;** — predecessor skips over the target node.
6. If curr == tail: **tail = prev;** — update tail pointer.
7. **delete curr; size--;**

```

else if ( opr == 3){
    cout<<"\n\nDelete SONG From Given Position . \n\n";

    if ( START == nullptr)
    {

```

```

        cout<<"PlayList is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<" ) - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 2 ; i < pos ; i++)
        {
            TEMP = TEMP->NEXT;
        }
        if (TEMP->NEXT == END){
            NODE *T1 = END;
            TEMP->NEXT = nullptr;
            END = TEMP;
        }
        else{
            NODE *T1 = TEMP->NEXT;
            TEMP->NEXT = T1->NEXT;
            delete(T1);
        }
    }
    countNode--;
    cout<<"\nSONG Deleted Successfully\n\n";
}

```

Traversal — $O(n)$

Singly Linked List – Traversal , search , count & Reverse

Traversal – $o(n)$

- I. Start curr = head
- II. Loop : while curr! = null
 - a. Print curr.data
 - b. Curr = curr.next
- III. End :- print 'NULL'
- IV. Output s4 – s3 – s2 – s1 - NULL

Search – $o(n)$ always

- I. Curr = head.scanned = 0
- II. While curr != NULL;
- III. Scanned ++
- IV. If curr.data has keyword
- V. Return curr <- FOUND
- VI. Curr = curr.next

COUNT – $0(1)$ best practice

- I. Count = 0 : curr = head
- II. While curr= NULL: count++

Method 2(maintain counter)

- I. Class has iont size = 0
- II. Insert node(): size++
- III. deleteNode () size --

Reverse – $o(n)$, $o(1)$ space

- I. Prev = NULL
- II. While curr != NULL
- III. Curr = curr.next <- save
- IV. Curr.next = prev <- filp
- V. Prev = curr



Concept

Traversal visits every node exactly once, following next pointers from head until NULL. This is how SONG renders your MANAGEMENT from top to bottom — each Song displayed in order.

10. **curr = head;** — start at newest Song.
11. Loop: **while (curr != nullptr)** — stop when we fall off the end.
12. Inside loop: process curr->data; then **curr = curr->next;**

```

void showNode()
{
    cout<<"\n\nSONG Detail -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nSONG Detail is empty\n";
    }
    else{
        while ( TEMP!= nullptr){
            cout<<"["<<TEMP->SID<<" ";
            cout<<TEMP->ANAME<<" ";
            cout<<TEMP->SD<<" ] -> ";
            TEMP = TEMP->NEXT;
        }
        cout<<"Null";
    }
}

```

Search by Keyword — $O(n)$

Why $O(n)$ only: Linked lists have no index formula. To find a node you must start at head and examine each node one by one. Best case $O(1)$ if found at head. Worst case $O(n)$ if at tail or absent.

Why binary search is impossible: Binary search requires jumping to the middle element. In a linked list, reaching node i requires following i pointers from head — $O(i)$. So finding the middle costs $O(n/2)$, making binary search $O(n \log n)$ — worse than linear $O(n)$.

```

int searchTweetInNode()
{
    cout<<"\n\nSearch SONG in PlayList -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\n Playlist is empty\n";
        return 0;
    }
    else{
        cout<<"\n\nEnter ARTIST Name To Find DETAIL -- ";

```

```

string Aname;
int counter = 0;
getline(cin,Aname);
while ( TEMP!= nullptr){
    if (TEMP->ANAME == Aname)
    {
        cout<<"\n["<<TEMP->SID<<" , ";
        cout<<TEMP->ANAME<<" , ";
        cout<<TEMP->SD<<"]\n";
        counter++;
    }
    TEMP = TEMP->NEXT;
}
if ( counter == 0){
    cout<<"\nNo SONG Found..\n\n";
}
}

```

Count Nodes — O(1) Best Practice

Approach	Algorithm	Time	Recommendation
Traverse and count	curr=head; while curr≠NULL: count++; curr=curr.next	O(n) per call	Avoid for frequent calls
Maintain size counter	size++ on insert, size-- on delete, getCount() returns size	O(1) per call	Always use this approach

```

void countNodes()
{
    system("cls");
    cout<<"Count SONG Module --\n\n";
    cout<<"Total Number of SONG -- "<<countNode<<" SONG Available\n\n";
}

```

Part 2: Doubly Linked List — Insertion, Deletion & Traversal

What the prev Pointer Adds

A doubly linked list adds one extra pointer — prev — to every node. This single addition costs 8 bytes per node on a 64-bit system but unlocks three major advantages:

- **O(1) delete of any node** — both neighbours are immediately accessible via prev and next. No scanning needed.
- **O(1) tail deletion** — tail.prev gives the second-to-last node directly. No traversal.
- **Backward traversal** — start at tail, follow prev pointers. Impossible in singly linked lists.

C++ Node Structure for Doubly Linked List

```
class NODED
{
public:
    NODED *PREV;
    string ID;
    string ANAME;
    string SONG;
    NODED *NEXT;
    NODED(string x, string y, string z)
    {
        PREV = nullptr;
        ID = x;
        ANAME = y;
        SONG = z;
        NEXT = nullptr;
    }
};

class DLinkedList
{
public:
    string SID;
```

```

int SCNT;
string ANAME, USONG;
NODED *START,*END;
int countNode;
DLinkedList(){
    SCNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}
void getData()
{
    SID = "SONG"+ to_string(SCNT);
    cout<<"\nEnter SID -- "<<SID<<endl;
    //getline(cin,SID);
    cout<<"Enter ARTIST Name -- ";
    //cin.ignore();
    getline(cin,ANAME);
    cout<<"Enter SONG -- ";
    getline(cin,USONG);
}

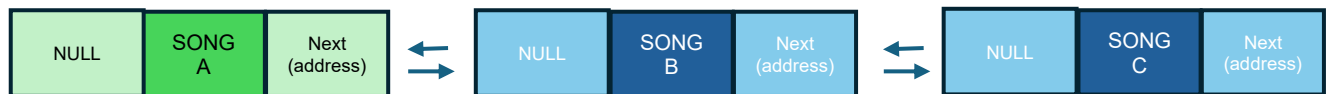
```

INSERT AT HEAD

BEFORE :-



AFTER :-

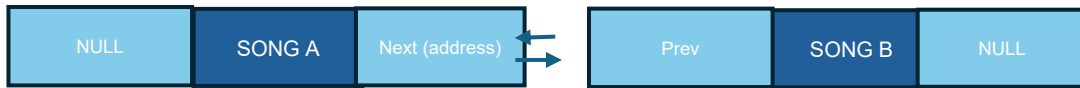


Steps :-

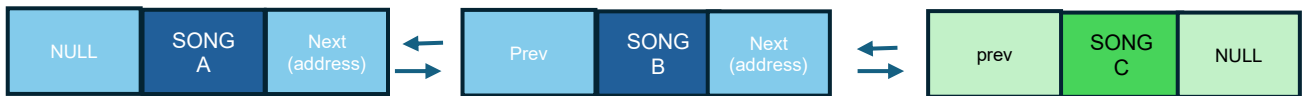
- 1 . create newNode
2. newNode.next = head(B)
- 3 . head. Prev = newNode <- key
4. head = newNode

INSERT AT END

BEFORE :-



AFTER :-



Step :-

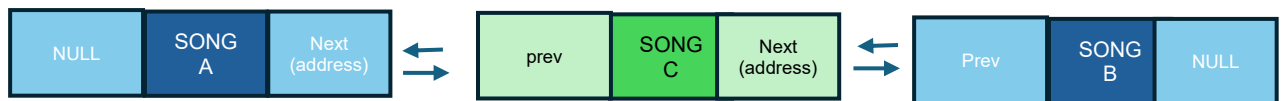
1. Create newNode
2. newNode.prev = tail(B) <- key !
3. tail.next = newNode
4. tail = newNode

INSERT AT GIVEN POSITION

BEFORE :-



AFTER :-



step

1. newNode.prev = A
2. newNode.next = B
3. A.next = newNode
4. B.prev = newNode <- doubly needs this extra step

Extra Step vs Singly

Compared to singly insertion at head, doubly adds one step: setting the old head's prev pointer back to the new node.

Algorithm — 4 pointer operations

1. **Create:** `SongNodeDL* newNode = new SongNodeDL(text, user);` — prev=NULL, next=NULL.
2. If empty: `head = tail = newNode;` return.
3. **Connect forward:** `newNode->next = head;` — new node points to old head.
4. **Connect backward:** `head->prev = newNode;` — old head points BACK to new node. ← extra step vs singly!
5. **Advance head:** `head = newNode;`
6. `size++;`

```
int insertNode(int opr)
{
    system("cls");
    if (opr == 1){
        cout<<"\n\nInsert SONG At Beginning . \n\n";
        getData();
        NODED *TEMP = new NODED(SID, ANAME, USONG);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if (START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START->PREV = TEMP;
            START = TEMP;
        }
        cout<<END->ANAME;
        countNode++;
        SCNT++;
    }
}
```

Insert at Tail — $O(1)$

Always $O(1)$: Doubly linked lists always maintain a tail pointer. Appending is direct — no traversal required in either the empty or non-empty case.

Algorithm — 4 pointer operations

1. **Create:** `SongNodeDL* newNode = new SongNodeDL(text, user);`
2. If empty: `head = tail = newNode;` return.
3. **Connect backward:** `newNode->prev = tail;` — new node looks BACK at current tail.
4. **Connect forward:** `tail->next = newNode;` — current tail points forward to new node.
5. **Advance tail:** `tail = newNode;`
6. `size++;`

```
else if ( opr == 2){
    cout<<"\n\nInsert  END At End . \n\n";
    getData();
    NODED *TEMP = new NODED(SID,ANAME, USONG);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    countNode++;
    SCNT++;
}
```

Insert at Position k — $O(k)$

Middle case — 4 pointer rewires (vs 2 in singly)

Traverse to position k-1 (same as singly), but now must wire FOUR pointers: both neighbours' prev and next must be updated.

```
else if ( opr == 3){
    cout<<"\n\nInsert SONG At Given Position . \n\n";
    cout<<"Total Nodes -- "<<countNode<<"\n\n";
    int pos;
    getData();
    cout<<"Enter SONG Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODED *TEMP = new NODED(SID,ANAME, USONG);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{

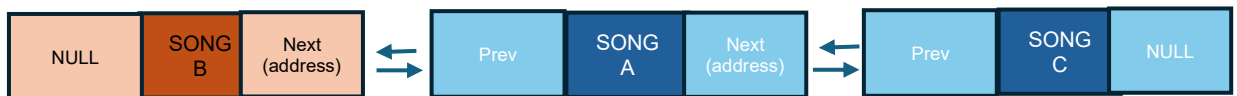
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV=TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    SCNT++;
}
```

Delete Head — O(1)

DELETE AT HEAD

DELETE HEAD – O(1)

1. Head = head.next
2. Head.prev = NULL <-
new head has no pred
3. Delete old_head
4. If head == NULL tail =
NULL



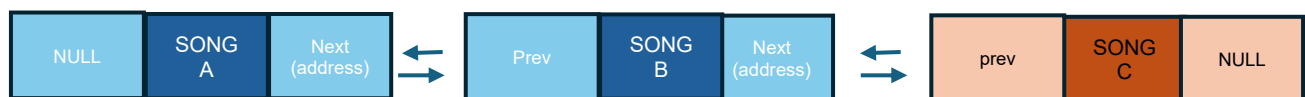
BEFORE :-



DELETE AT TAIL

DELETE TAIL – O(1)

1. Tail = tail.prev <- via prev
pointer
2. Tail.next = NULL
3. Delete old_tail
4. If tail == NULL ,head =
NULL



AFTER :-

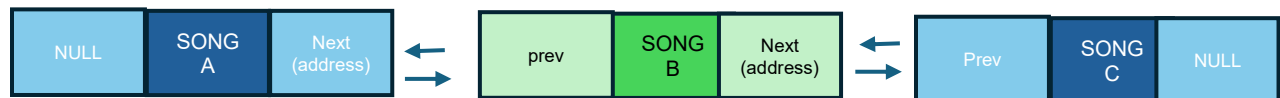


DELETE AT GIVEN POSITION

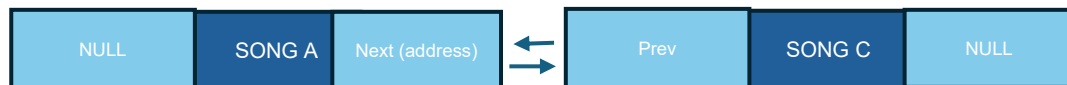
DELETE AT POSITION- 0(1)

1. $N.\text{prev.next} = n.\text{next}$ (A skip over N)
2. $N.\text{next.prev} = N.\text{prev}$ (B pointer back to A)
3. Delete N

BEFORE :-



AFTER :-



Algorithm

1. **Check:** If $\text{head} == \text{NULL}$: empty list, return.
2. **Save:** $\text{SongNodeDL}^* \text{temp} = \text{head};$
3. **Advance head:** $\text{head} = \text{head} \rightarrow \text{next};$
4. If $\text{head} \neq \text{NULL}$: $\text{head} \rightarrow \text{prev} = \text{nullptr};$ — new head has no predecessor.
5. Else (list now empty): $\text{tail} = \text{nullptr};$
6. **Free:** $\text{delete temp}; \text{size--};$

Delete node at head

```
int deleteNode(int opr)
{
    NODED *TEMP;
    if (opr == 1){
        cout<<"\n\nDelete SONG From Beginning . \n\n";
        if (START == nullptr)
        {
            cout<<"Playlist is Empty.. \n\n";
            return 0;
        }
        else if (START->NEXT == nullptr){
            TEMP = START;
```

```

        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
        START->PREV=nullptr;
    }
    countNode--;
    delete(TEMP);
    cout<<"\n SONG Node Deleted Successfully\n\n";
}

```

Delete Tail — O(1) ★ Key Advantage Over Singly

Why This is Doubly's Key Advantage

In a singly linked list, deleting the tail requires $O(n)$ traversal to find the second-to-last node (which must have its next set to NULL). In a doubly linked list, `tail.prev` ALREADY points to the second-to-last node — $O(1)$, no scanning!

Algorithm — 3 pointer operations

1. **Check:** If `tail == NULL`: empty list, return.
2. **Save:** `SongNodeDL* temp = tail;`
3. **Advance tail BACKWARD:** `tail = tail->prev;` ← ONLY possible because of the prev pointer!
4. If `tail != NULL`: `tail->next = nullptr;` — new tail has no successor.
5. Else (list now empty): `head = nullptr;`
6. `delete temp; size--;`

```

cout<<"\n\nDelete SONG From End . \n\n";
if ( START == nullptr)
{
    cout<<"Playlist is Empty.. \n\n";
    return 0;
}
else if ( START->NEXT == nullptr){
    TEMP = START;
    END = START = nullptr;
}
else{
    NODED *TEMP = END;
    END = END->PREV;
}

```

```

    END->NEXT = nullptr;
    delete(TEMP);
}
countNode--;
cout<<"\n SONG Node Deleted Successfully\n\n";
}

```

Delete given position Node — O(1) with pointer / O(n) if search required

The Biggest Advantage

If you already have a direct pointer to the node to delete, O(1) is achievable because both neighbours are accessible via prev and next. Just 2 rewires bypass the node completely.

Algorithm — 2 pointer rewires

1. Check edge cases: if $N == \text{head}$ → call `deleteHead()`; if $N == \text{tail}$ → call `deleteTail()`.
2. **A.next = N.next:** $N \rightarrow \text{prev} \rightarrow \text{next} = N \rightarrow \text{next}$; — predecessor skips over N.
3. **B.prev = N.prev:** $N \rightarrow \text{next} \rightarrow \text{prev} = N \rightarrow \text{prev}$; — successor points back past N.
4. **delete N; size--;**

```

else if ( opr == 3){
    cout<<"\n\nDelete SONG From Given Position . \n\n";

    if ( START == nullptr)
    {
        cout<<"Playlist is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 1 ; i < pos ; i++)
        {

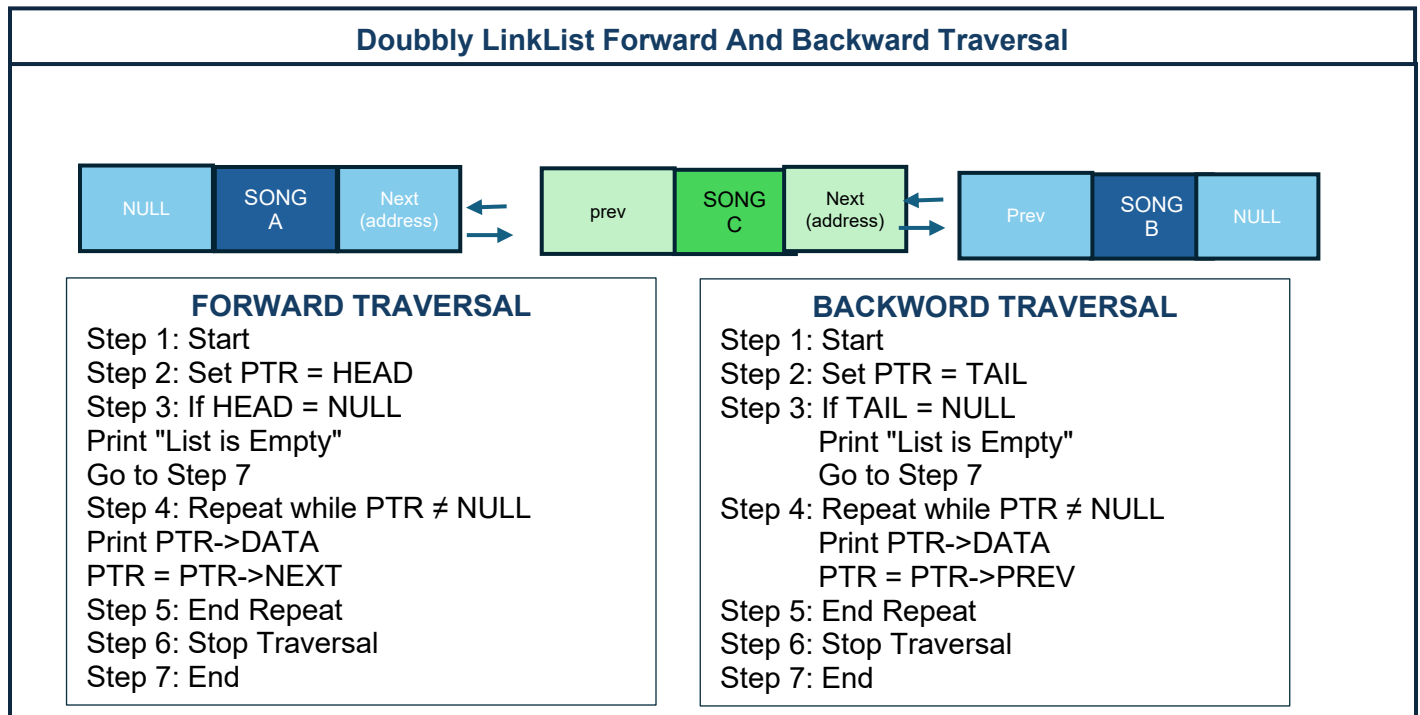
```

```

        TEMP = TEMP->NEXT;
    }
    if (TEMP == END){
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
        delete(TEMP);
    }
    else{
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countNode--;
cout<<"\n SONG Node Deleted Successfully\n\n";
}

```

Forward & Backward Traversal — $O(n)$ each



Forward Traversal (Newest → Oldest)

```
int showNode()
{
    cout<<"\n\nDoubly Link List Traversal Of SONGS -- \n\n";
    cout<<"* Head -> ";
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
        return 0;
    }
    cout<<"\n\n1. Forward SONG Traversal..\n";
    cout<<"2. Backward SONG Traversal\n";
    cout<<"3. Goto to Main Menu..\n";
    cout<<"4. Exit\n";
    char c = getch();
    if ( c == '1'){
```

```

cout<<"\n\nForward Traversal\n\n";
cout<<"null <- ";
NODED *TEMP = START;
while ( TEMP!= nullptr){
    cout<<"["<<TEMP->ID<<" ";
    cout<<TEMP->ANAME<<" ";
    cout<<TEMP->SONG<<"] -> ";
    TEMP = TEMP->NEXT;
}
cout<<"Null\n\n";
}

```

Backward Traversal (Oldest → Newest) ← Impossible in Singly

Unique to doubly linked lists. Start at tail, follow prev pointers. This direction is physically impossible in a singly linked list because there is no prev pointer.

```

else if ( c == '2'){
    cout<<"\n\nBackward SONG Traversal\n\n";
    cout<<"null <- ";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->ANAME<<" ";
        cout<<TEMP->SONG<<"] -> ";
        TEMP = TEMP->PREV;
    }
    cout<<"Null\n\n";
}
else if ( c == '3'){
    MainMenu();
}
else if (c == '4'){
    exit(0);
}

```

}

}

Real-World Applications of Backward Traversal

Application	How Backward Traversal is Used
Browser back button	Navigate history in reverse (most recent → oldest visited)
Text editor undo/redo	Move backward through change history to undo actions
Music player Previous Track	Go backward through the playlist
LRU Cache eviction	Tail is LRU item — accessed via backward from end, evicted $O(1)$
Twitter / Social Media	Show thread in chronological order (oldest reply first)

This code is the main module of a C++ based Music Management System Project that uses both Singly Linked List and Doubly Linked List concepts. It acts like the starting menu (main controller) of the whole project.

CODE OF MAIN MODULE

```
#include<iostream>
#include<conio.h>
#include"singlylinklist.cpp"
#include"dubbllylinklist.cpp"
class MainLinkList
{
public:
    LinkList l;
    DLinkList dl;
    void MainLinkListModule()
    {
        system("cls");
        cout<<"----- WELCOME MUSIC SYSTEM PROJECT -----\\n\\n";
        cout<<"1. Singly Link List \\n\\n";
        cout<<"2. Doubly Link List \\n\\n";
        cout<<"3. Exit \\n\\n";
        cout<<"Choose any one of them....";
        char c = getch();
        if ( c == '1'){
            l.MainMenu();
        }
        else if ( c == '2'){
            dl.MainMenu();
        }
        else if ( c == '3'){
            exit(0);
        }
    }
};
int main()
{
    MainLinkList mll;
    mll.MainLinkListModule();
    cout<<"\\n\\nThank You for Using My App..\\m\\m";
    cout<<"Design & Coded By – Meenakshi Mali .. \\n\\n";
}
```

Output

```
----- WELCOME MUSIC SYSTEM PROJECT -----  
1. Singly Link List  
2. Doubly Link List  
3. Exit  
Choose any one of them....|
```

Final code SinglyLinkedList

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODE
{
public:
    string SID;
    string ANAME;
    string SD;
    NODE *NEXT;
    NODE(string x, string y, string z)
    {
        SID = x;
        ANAME = y;
        SD = z;
        NEXT = nullptr;
    }
};
class LinkList
{
public:
    string SID;
    int SCNT;
    string AName, SCAP;
    NODE *START,*END;
    int countNode;
```

```

LinkedList(){
    SCNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}

void getData()
{
    SID = "SONG"+ to_string(SCNT);
    cout<<"\nGENERATE SONG ID -- "<<SID<<endl;
    cout<<"Enter ARTIST NAME -- ";
    getline(cin,AName);
    cout<<"Enter SONG Name -- ";
    getline(cin,SCAP);
}

char continueCheck()
{
    char c;
    cout<<"\n\nAre you.. \n";
    cout<<"1. Continue with Same\n";
    cout<<"2. Go to Main Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
        exit(0);
    }else{
        return (c);
    }
}

```

```

int insertNode(int opr)
{
    system("cls");
    if ( opr == 1){
        cout<<"\n\nInsert SONG At Beginning . \n\n";
        getData();
        NODE *TEMP = new NODE(SID,AName, SCAP);
        if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if ( START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START = TEMP;
        }
        countNode++;
        SCNT++;
    }
    else if ( opr == 2){
        cout<<"\n\nInsert SONG At End . \n\n";
        getData();
        NODE *TEMP = new NODE(SID,AName, SCAP);

        if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if ( START == nullptr){

```

```

        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        END = TEMP;
    }
    countNode++;
    SCNT++;
}
else if ( opr == 3){
    cout<<"\n\nInsert SONG At Given Position . \n\n";
    cout<<"Total Songs -- "<<countNode<<"\n\n";
    int pos;
    getData();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODE *TEMP = new NODE(SID,AName, SCAP);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{

```

```

        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    SCNT++;
}
char c = continueCheck();
if (c == '1')
    insertNode(opr);
else if ( c == '2')
    insertionOperation();
else if ( c == '3')
    MainMenu();
}
void showNode()
{
    cout<<"\n\nSONG Detail -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nSONG Detail is empty\n";
    }
    else{
        while ( TEMP!= nullptr){
            cout<<"["<<TEMP->SID<<" ";
            cout<<TEMP->ANAME<<" ";
            cout<<TEMP->SD<<"] -> ";
            TEMP = TEMP->NEXT;
        }
    }
}

```

```

    }
    cout<<"Null";
}
}

void countNodes()
{
    system("cls");
    cout<<"Count SONG Module --\n\n";
    cout<<"Total Number of SONG -- "<<countNode<<" SONG Available\n\n";
}

int insertionOperation()
{
    system("cls");
    cout<<"\nInsertion Operation..\n\n";
    cout<<"1. Insert SONG At Beginning\n";
    cout<<"2. Insert SONG At End\n";
    cout<<"3. Insert SONG After Data\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        insertNode(1);
    }
    else if ( c == '2'){
        insertNode(2);
    }
    else if ( c == '3'){
        insertNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
}

```

```

    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}

int deleteNode(int opr)
{
    NODE *TEMP;
    if ( opr == 1){
        cout<<"\n\nDelete SONG From Beginning . \n\n";
        if ( START == nullptr)
        {
            cout<<"PlayList is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{
            TEMP = START;
            START = START->NEXT;
        }
        countNode--;
        delete(TEMP);
        cout<<"\nSONG Deleted Successfully\n\n";
    }
    else if ( opr == 2){
        cout<<"\n\nDelete SONG From End . \n\n";
    }
}

```

```

if ( START == nullptr)
{
    cout<<"PlayList is Empty.. \n\n";
    return 0;
}
else if ( START->NEXT == nullptr){
    TEMP = START;
    END = START = nullptr;
}
else{
    TEMP = START;
    int c = 1;
    while ( TEMP->NEXT != END)
    {
        cout<<c++<<endl;
        TEMP = TEMP->NEXT;
    }
    NODE *T1 = END;
    TEMP->NEXT = nullptr;
    END = TEMP;
    cout<<END->ANAME<<endl;
    delete(T1);
}
countNode--;
cout<<"\nSONG Deleted Successfully\n\n";
}
else if ( opr == 3){
    cout<<"\n\nDelete SONG From Given Position . \n\n";

    if ( START == nullptr)
    {

```

```

        cout<<"PlayList is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 2 ; i < pos ; i++)
        {
            TEMP = TEMP->NEXT;
        }
        if (TEMP->NEXT == END){
            NODE *T1 = END;
            TEMP->NEXT = nullptr;
            END = TEMP;
        }
        else{
            NODE *T1 = TEMP->NEXT;
            TEMP->NEXT = T1->NEXT;
            delete(T1);
        }
    }
    countNode--;
    cout<<"\nSONG Deleted Successfully\n\n";
}

```

```

char c = continueCheck();
if (c == '1')
    deleteNode(opr);
else if ( c == '2')
    deleteOperation();
else if ( c == '3')
    MainMenu();
    return 0;
}
int deleteOperation()
{
    system("cls");
    cout<<"\nDeletion Operation..\n\n";
    cout<<"1. Delete SONG From Beginning\n";
    cout<<"2. Delete SONG From End\n";
    cout<<"3. Delete SONG From Given Position\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        deleteNode(1);
    }
    else if ( c == '2'){
        deleteNode(2);
    }
    else if ( c == '3'){
        deleteNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
}

```

```

else if ( c == '5'){
    thanksLine();
    exit(0);
}

}

/*search module start*/

int searchTweetInNode()
{
    cout<<"\n\nSearch SONG in  PlayList -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\ Playlist is empty\n";
        return 0;
    }
    else{
        cout<<"\n\nEnter ARTIST Name To Find DETAIL -- ";
        string Aname;
        int counter = 0;
        getline(cin,Aname);
        while ( TEMP!= nullptr){
            if (TEMP->ANAME == Aname)
            {
                cout<<"\n["<<TEMP->SID<<" , ";
                cout<<TEMP->ANAME<<" , ";
                cout<<TEMP->SD<<"]\n";
                counter++;
            }
            TEMP = TEMP->NEXT;

```

```

    }
    if ( counter == 0){
        cout<<"\nNo SONG Found..\n\n";
    }
}

char c;
cout<<"\n\nAre you .. \n";
cout<<"1. Continue with Same\n";
cout<<"2. Go to Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    searchTweetInNode();
else if ( c == '2')
    MainMenu();
else if ( c == '3'){
    thanksLine();
    exit(0);

}

return 0;
}

/*Search Module End*/
int MainMenu()
{
    system("cls");
    cout<<"===== PLAYLIST MENU =====\n\n";
    cout<<"1. Insertion Of SONG \n";
    cout<<"2. Traversal SONG\n";
    cout<<"3. Delete SONG \n";

```

```

cout<<"4. Search SONG\n";
cout<<"5. Count SONG\n";
cout<<"6. Exit\n";
cout<<"Choose One of them -- ";
char c = getch();
if ( c == '1'){
    insertionOperation();
}
else if ( c == '2'){
    showNode();
}
else if ( c == '3'){
    deleteOperation();
}
else if ( c == '4'){
    searchTweetInNode();
}
else if ( c == '5'){
    countNodes();
}
else if ( c == '6'){
    thanksLine();
    exit(0);
}
cout<<"\n\nDo you want to continue .. press 1 else any key..";
c = getch();
if ( c == '1'){
    MainMenu();
}
}
void thanksLine()

```

```

{
    cout<<"\n\nThanks for Using My APP\n\n";
}
};

/*int main()
{
    LinkList l;
    l.MainMenu();
    cout<<"\n\nThanks for Using My APP\n\n";
}*/

```

Output

```

===== PLAYLIST MENU =====
1. Insertion Of SONG
2. Traversal SONG
3. Delete SONG
4. Search SONG
5. Count SONG
6. Exit
Choose One of them --

```

```

n Insertion Operation..
1. Insert SONG At Beginning
2. Insert SONG At End
3. Insert SONG After Data
4. Go to Main Menu
5. Exit.
|

```

```
Insert SONG At Beginning .

GENERATE SONG ID -- SONG1001
Enter ARTIST NAME  -- Arijit singh
Enter SONG Name -- ABCD
```

```
Are you..
1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit
|
```

```
===== PLAYLIST MENU =====
n
1. Insertion Of SONG
2. Traversal SONG
3. Delete SONG
4. Search SONG
5. Count SONG
6. Exit
Choose One of them --

SONG Detail --

* Head -> [SONG1002, akshita, cdf] -> [SONG1001, Arijit singh, ABCD] -> Null

Do you want to continue .. press 1 else any key..|
```

Deletion Operation..

1. Delete SONG From Beginning
2. Delete SONG From End
3. Delete SONG From Given Position
4. Go to Main Menu
5. Exit.

Delete SONG From Beginning .

SONG Deleted Successfully

Are you..

1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit

Dubbly LinkList

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODED
{
public:
    NODED *PREV;
    string ID;
    string ANAME;
    string SONG;
    NODED *NEXT;
    NODED(string x, string y, string z)
    {
        PREV = nullptr;
        ID = x;
        ANAME = y;
        SONG = z;
        NEXT = nullptr;
    }
};
class DLinkedList
{
public:
    string SID;
    int SCNT;
    string ANAME, USONG;
    NODED *START,*END;
```

```

int countNode;
DLinkedList(){
    SCNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}
void getData()
{
    SID = "SONG"+ to_string(SCNT);
    cout<<"\nEnter SID -- "<<SID<<endl;
    //getline(cin,SID);
    cout<<"Enter ARTIST Name -- ";
    //cin.ignore();
    getline(cin,ANAME);
    cout<<"Enter SONG -- ";
    getline(cin,USONG);
}
char continueCheck()
{
    char c;
    cout<<"\n\nAre you.. \n";
    cout<<"1. Continue with Same\n";
    cout<<"2. Go to Main Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
        exit(0);
    }else{

```

```

        return (c);
    }
}

int insertNode(int opr)
{
    system("cls");
    if ( opr == 1){
        cout<<"\n\nInsert  SONG At Beginning . \n\n";
        getData();
        NODED *TEMP = new NODED(SID,ANAME, USONG);
        if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if ( START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START->PREV = TEMP;
            START = TEMP;
        }
        cout<<END->ANAME;
        countNode++;
        SCNT++;
    }
    else if ( opr == 2){
        cout<<"\n\nInsert  END At End . \n\n";
        getData();
        NODED *TEMP = new NODED(SID,ANAME, USONG);
        if ( TEMP == nullptr){

```

```

        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    countNode++;
    SCNT++;
}
else if ( opr == 3){
    cout<<"\n\nInsert SONG At Given Position . \n\n";
    cout<<"Total Nodes -- "<<countNode<<"\n\n";

    int pos;
    getData();
    cout<<"Enter SONG Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODED *TEMP = new NODED(SID,ANAME, USONG);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
}

```

```

    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{

        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV=TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    SCNT++;
}
char c = continueCheck();
if (c == '1')
    insertNode(opr);
else if ( c == '2')
    insertionOperation();
else if ( c == '3')
    MainMenu();
}
int showNode()
{
    cout<<"\n\nDoubly Link List Traversal Of SONGS -- \n\n";
    cout<<"* Head -> ";
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
    }
}

```

```

    return 0;
}
cout<<"\n\n1. Forward SONG Traversal..\n";
cout<<"2. Backward SONG Traversal\n";
cout<<"3. Goto to Main Menu..\n";
cout<<"4. Exit\n";
char c = getch();
if ( c == '1'){
    cout<<"\n\nForward Traversal\n\n";
    cout<<"null <- ";
    NODED *TEMP = START;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->ANAME<<" ";
        cout<<TEMP->SONG<<" ] -> ";
        TEMP = TEMP->NEXT;
    }
    cout<<"Null\n\n";
}
else if ( c == '2'){
    cout<<"\n\nBackward SONG Traversal\n\n";
    cout<<"null <- ";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->ANAME<<" ";
        cout<<TEMP->SONG<<" ] -> ";
        TEMP = TEMP->PREV;
    }
    cout<<"Null\n\n";
}

```

```

else if ( c == '3'){
    MainMenu();
}
else if (c == '4'){
    exit(0);
}

}

void countNodes()
{
    system("cls");
    cout<<"Count SONG Nodes Module --\n\n";
    cout<<"Total Number of SONG In nodes -- "<<countNode<<" SONG Nodes Available\n\n"
; }

int insertionOperation()
{
    system("cls");
    cout<<"\nInsertion Operation..\n\n";
    cout<<"1. Insert SONG At Beginning\n";
    cout<<"2. Insert SONG At End\n";
    cout<<"3. Insert SONG After Data\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        insertNode(1);
    }
    else if ( c == '2'){
        insertNode(2);
    }
    else if ( c == '3'){

```

```

        insertNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}

int deleteNode(int opr)
{
    NODED *TEMP;
    if ( opr == 1){
        cout<<"\n\nDelete SONG From Beginning . \n\n";
        if ( START == nullptr)
        {
            cout<<"Playlist is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{
            TEMP = START;
            START = START->NEXT;
            START->PREV=nullptr;
        }
        countNode--;
    }
}

```

```

delete(TEMP);
cout<<"\n SONG Node Deleted Successfully\n\n";
}
else if ( opr == 2){
    cout<<"\n\nDelete SONG From End . \n\n";
    if ( START == nullptr)
    {
        cout<<"Playlist is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        NODED *TEMP = END;
        END = END->PREV;
        END->NEXT = nullptr;
        delete(TEMP);
    }
    countNode--;
    cout<<"\n SONG Node Deleted Successfully\n\n";
}
else if ( opr == 3){
    cout<<"\n\nDelete SONG From Given Position . \n\n";

    if ( START == nullptr)
    {
        cout<<"Playlist is Empty.. \n\n";
        return 0;
    }
}

```

```

int pos;
cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
cin>>pos;
if ( pos == 1 && START == END)
{
    TEMP = START;
    START = END = nullptr;
}
else{
    TEMP = START;
    for (int i = 1 ; i < pos ; i++)
    {
        TEMP = TEMP->NEXT;
    }
    if (TEMP == END){
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
        delete(TEMP);
    }
    else{
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countNode--;
cout<<"\n SONG Node Deleted Successfully\n\n";
}
char c = continueCheck();
if (c == '1')
    deleteNode(opr);

```

```

else if ( c == '2')
    deleteOperation();
else if ( c == '3')
    MainMenu();
}
int deleteOperation()
{
    system("cls");
    cout<<"\nDeletion Operation..\n\n";
    cout<<"1. Delete SONG From Beginning\n";
    cout<<"2. Delete SONG From End\n";
    cout<<"3. Delete SONG From Given Position\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        deleteNode(1);
    }
    else if ( c == '2'){
        deleteNode(2);
    }
    else if ( c == '3'){
        deleteNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}

```

```

}

/*search module start*/

int searchSONGInNode()
{
    cout<<"\n\nSearch SONG in Nodes -- \n\n";
    cout<<"* Head -> ";
    NODED *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
        return 0;
    }
    else{
        cout<<"\n\nEnter User Name To Find SONGS -- ";
        string ANAME;
        int counter = 0;
        getline(cin,ANAME);
        while ( TEMP!= nullptr){
            if (TEMP->ANAME == ANAME)
            {
                cout<<"\n["<<TEMP->ID<<" , ";
                cout<<TEMP->ANAME<<" , ";
                cout<<TEMP->SONG<<"]\n";
                counter++;
            }
            TEMP = TEMP->NEXT;
        }
        if ( counter == 0){
            cout<<"\nNo SONG Found..\n\n";
        }
    }
}

```

```

char c;
cout<<"\n\nAre you .. \n";
cout<<"1. Continue with Same\n";
cout<<"2. Go to Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    searchSONGInNode();
else if ( c == '2')
    MainMenu();
else if ( c == '3'){
    thanksLine();
    exit(0);
}
}

/*Search Module End*/
int MainMenu()
{
    system("cls");
    cout<<"===== PLAYLIST MENU =====\n\n";
    cout<<"1. Insertion Operation\n";
    cout<<"2. Traversal Operation\n";
    cout<<"3. Delete Operation\n";
    cout<<"4. Search Operation\n";
    cout<<"5. Count Nodes\n";
    cout<<"6. Exit\n";
    cout<<"Choose One of them -- ";
    /*l.insertNode(101,"Pooja","Hello Friends");
    l.insertNode(102,"Niharika","Hello Pooja");
    l.insertNode(103,"Deepak","Hello Tejswi");

```

```

l.showNode();*/
char c = getch();
if ( c == '1'){
    insertionOperation();
}
else if ( c == '2'){
    showNode();
}
else if ( c == '3'){
    deleteOperation();
}
else if ( c == '4'){
    searchSONGInNode();
}
else if ( c == '5'){
    countNodes();
}
else if ( c == '6'){
    thanksLine();
    exit(0);
}
cout<<"\n\nDo you want to continue .. press 1 else any key..";
c = getch();
if ( c == '1'){
    MainMenu();
}
}
void thanksLine()
{
    cout<<"\n\nThanks for Using My APP\n\n";
}

```

```
};

/*int main()
{
    DLinkedList l;
    l.MainMenu();
    cout<<"\n\nThanks for Using My APP\n\n";
}*/
```

output

```
===== PLAYLIST MENU =====
1. Insertion Operation
2. Traversal Operation
3. Delete Operation
4. Search Operation
5. Count Nodes
6. Exit
Choose One of them -- |
```

```
Insertion Operation..
1. Insert SONG At Beginning
2. Insert SONG At End
3. Insert SONG After Data
4. Go to Main Menu
5. Exit.
|
```

```
Insert SONG At Beginning .
```

```
Enter SID -- SONG1001
```

```
Enter ARTIST Name -- ABCD
```

```
Enter SONG -- EFh
```

```
ABCD
```

```
Are you..
```

```
1. Continue with Same
```

```
2. Go to Main Previous Menu
```

```
3. Go to Main Menu..
```

```
4. Exit
```

```
|
```

```
Doubly Link List Traversal Of SONGS --
```

```
* Head ->
```

```
1. Forward SONG Traversal..
```

```
2. Backward SONG Traversal
```

```
3. Goto to Main Menu..
```

```
4. Exit
```

```
Forward Traversal
```

```
null <- [SONG1001, ABCD, EFh] -> Null
```

```
Process returned -1073741795 (0xC000001D) execution time : 84.234 s
```

```
Press any key to continue.
```

```
|
```

```
===== PLAYLIST MENU =====
```

1. Insertion Operation
2. Traversal Operation
3. Delete Operation
4. Search Operation
5. Count Nodes
6. Exit

Choose One of them --

Search SONG in Nodes --

* Head ->

Enter User Name To Find SONGS -- akshita

[SONG1001, akshita, abcd]

Are you ..

1. Continue with Same
2. Go to Main Menu..
3. Exit

|

Deletion Operation..

1. Delete SONG From Beginning
2. Delete SONG From End
3. Delete SONG From Given Position
4. Go to Main Menu
5. Exit.

Delete SONG From Beginning .

SONG Node Deleted Successfully

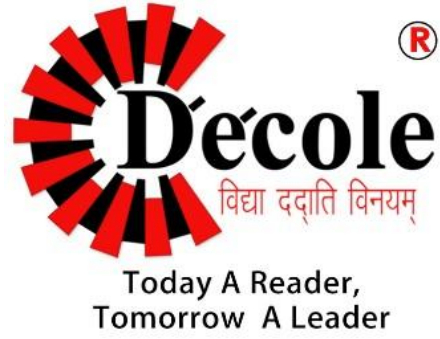
Are you..

1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit

|

Scan & Check My Project on GitHub





Thank You

Meenakshi MALi
B.C.A. 2nd Semester
Dezyne École College